EECS 348 Assignment 3

a. Name of two GenAI programs and b. How you accessed them (e.g., Google query, CoPilot on my computer)

chatgpt via https://chatgpt.com/
gemini via https://gemini.google.com/app

c. Prompt used. (You must use the same prompt for both GenAIs.)
Write a complete C++ program for a CEO email priority queue. Requirements:

DESIGN
- Use classes/objects, not free functions (main may only create objects and call them).
- Implement your own list-based (array/vector) MaxHeap from scratch. Do not use std::priority_queue, std::make_heap, or any pre-existing heap module.
- Use the MaxHeap as the priority queue for all email prioritizing.

PRIORITY
- Sender categories, highest to lowest: Boss, Subordinate, Peer, ImportantPerson, OtherPerson.
- Within the same category, the newest email (latest date) is read first.
- Dates are MM-DD-YYYY.

INPUT
- The input filename is passed as a command-line argument: ./program input.txt
- If the argument is missing, print "Usage: ./program <input_file>" and exit. If the file cannot be opened, print an error naming the file and exit.
- Process every line, including the last one, even if it has no trailing newline or has Windows (CRLF) line endings. Skip blank lines and trim whitespace.
- Commands:
  EMAIL <sender category>,<subject line>,<date> (EMAIL is followed by a space; fields are comma-delimited; the subject may contain spaces but not commas)
  NEXT - display the highest priority email without removing it
  READ - remove the highest priority email
  COUNT - display the number of unread emails

OUTPUT FORMAT
- COUNT prints: There are N emails to read.
- NEXT prints:
  Next email:
  Sender: <category>
  Subject: <subject>
  Date: <date>
- Two NEXTs in a row print the same email. Two READs in a row delete two emails without displaying them.

- NEXT or READ on an empty queue must not crash; NEXT prints "No emails to read." and READ prints nothing.

EXAMPLE
Input:
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT

Expected output:
There are 6 emails to read.
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025
Next email:
Sender: Boss
Subject: Important
Date: 12-20-2024
There are 4 emails to read.

CODE STYLE
- Start the file with a prologue comment: program name, description, inputs, outputs, author, creation date, sources (name the GenAI used).
- Comment every line and every major block of code.
- Must compile with g++ -std=c++17 on a Linux server.

d. Generated code from both GenAIs.
ChatGPT output:

```
/*******************************************************************************
 * Program Name: CEO Email Priority Queue
 *
 * Description:
 *     This program implements a priority queue for a CEO's emails using a
```

```
*     custom array-based MaxHeap. Emails are prioritized first by sender
*     category and then by date, with newer emails having higher priority
*     within the same sender category.
*
* Inputs:
*     A text file provided as a command-line argument.
*     Commands in the file are:
*         EMAIL <sender category>,<subject>,<date>
*         NEXT
*         READ
*         COUNT
*
* Outputs:
*     COUNT displays the number of unread emails.
*     NEXT displays the highest-priority unread email.
*     READ removes the highest-priority unread email.
*
* Author: Josselyn Bui
* Creation Date: 10/01/2026
*
* Sources:
*     OpenAI ChatGPT (GPT-5.6 Luna) - used for programming assistance.
*********************************************************************************/

#include <iostream>      // Provides input and output functionality.
#include <fstream>       // Provides file input functionality.
#include <string>        // Provides the string class.
#include <vector>        // Provides the vector class used for the heap.
#include <sstream>       // Provides string stream functionality.
#include <algorithm>     // Provides general string/algorithm utilities.

using namespace std;     // Allows standard library names to be used directly.

/*********************************************************************************
 * Function: trim
 *
 * Description:
 *     Removes whitespace from the beginning and end of a string.
 *
 * Parameters:
 *     str - The string that should be trimmed.
 *
 * Returns:
 *     A copy of the string with surrounding whitespace removed.
```

```
 ******************************************************************************/
string trim(const string& str)
{
    // Find the first character that is not whitespace.
    size_t first = str.find_first_not_of(" \t\r\n");

    // If the string contains only whitespace, return an empty string.
    if (first == string::npos)
    {
        return "";
    }

    // Find the last character that is not whitespace.
    size_t last = str.find_last_not_of(" \t\r\n");

    // Return the substring between the first and last non-whitespace characters.
    return str.substr(first, last - first + 1);
}

/*******************************************************************************
 * Class: Email
 *
 * Description:
 *     Represents one email. The class stores the sender category, subject,
 *     and date and provides a priority comparison between emails.
 ******************************************************************************/
class Email
{
private:
    // Stores the sender category of the email.
    string sender;

    // Stores the subject line of the email.
    string subject;

    // Stores the date of the email in MM-DD-YYYY format.
    string date;

    // Stores the numerical priority of the sender category.
    int senderPriority;

    /***************************************************************************
     * Function: calculateSenderPriority
     *
```

```
 * Description:
 *     Converts a sender category into its priority number.
 *
 * Returns:
 *     A larger number means higher priority.
 ******************************************************************************/
int calculateSenderPriority(const string& category)
{
    // Boss has the highest sender priority.
    if (category == "Boss")
    {
        return 5;
    }

    // Subordinate has the second-highest sender priority.
    if (category == "Subordinate")
    {
        return 4;
    }

    // Peer has the third-highest sender priority.
    if (category == "Peer")
    {
        return 3;
    }

    // ImportantPerson has the fourth-highest sender priority.
    if (category == "ImportantPerson")
    {
        return 2;
    }

    // OtherPerson has the lowest sender priority.
    return 1;
}

/******************************************************************************
 * Function: dateValue
 *
 * Description:
 *     Converts MM-DD-YYYY into a number that can be compared.
 *
 * Returns:
 *     A numeric representation of the date.
```

```
    ***************************************************************************/
    long long dateValue() const
    {
        // Extract the month from the date.
        int month = stoi(date.substr(0, 2));

        // Extract the day from the date.
        int day = stoi(date.substr(3, 2));

        // Extract the year from the date.
        int year = stoi(date.substr(6, 4));

        // Convert the date to YYYYMMDD so chronological comparison works.
        return static_cast<long long>(year) * 10000
            + static_cast<long long>(month) * 100
            + day;
    }

public:
    /***************************************************************************
     * Constructor: Email
     *
     * Description:
     *     Creates an Email object using the supplied information.
     ***************************************************************************/
    Email(const string& senderInput,
          const string& subjectInput,
          const string& dateInput)
    {
        // Store the sender category.
        sender = senderInput;

        // Store the subject.
        subject = subjectInput;

        // Store the date.
        date = dateInput;

        // Calculate and store the sender's priority.
        senderPriority = calculateSenderPriority(sender);
    }

    /***************************************************************************
     * Function: higherPriorityThan
```

```
 *
 * Description:
 *     Determines whether this email has a higher priority than another
 *     email.
 *
 * Parameters:
 *     other - The email being compared against.
 *
 * Returns:
 *     true if this email should appear before the other email.
 ***************************************************************************/
bool higherPriorityThan(const Email& other) const
{
   // Compare sender categories first.
   if (senderPriority != other.senderPriority)
   {
      // The larger sender priority wins.
      return senderPriority > other.senderPriority;
   }

   // If sender categories are equal, compare the dates.
   if (dateValue() != other.dateValue())
   {
      // The newer date has higher priority.
      return dateValue() > other.dateValue();
   }

   // If both category and date are equal, treat them as equal priority.
   return false;
}

/***************************************************************************
 * Function: getSender
 *
 * Description:
 *     Returns the sender category.
 ***************************************************************************/
string getSender() const
{
   // Return the sender category.
   return sender;
}

/***************************************************************************
```

```
     * Function: getSubject
     *
     * Description:
     *     Returns the email subject.
     ***************************************************************************/
    string getSubject() const
    {
        // Return the subject.
        return subject;
    }

    /***************************************************************************
     * Function: getDate
     *
     * Description:
     *     Returns the email date.
     ***************************************************************************/
    string getDate() const
    {
        // Return the date.
        return date;
    }
};

/*******************************************************************************
 * Class: MaxHeap
 *
 * Description:
 *     Implements a MaxHeap from scratch using a vector as an array.
 *     The highest-priority email is always stored at index 0.
 *******************************************************************************/
class MaxHeap
{
private:
    // Stores all emails in the heap's underlying array.
    vector<Email> heap;

    /***************************************************************************
     * Function: parent
     *
     * Description:
     *     Calculates the parent index of a node.
     ***************************************************************************/
    int parent(int index)
```

```
{
    // Return the parent index.
    return (index - 1) / 2;
}

/*************************************************************************
 * Function: leftChild
 *
 * Description:
 *     Calculates the left child index of a node.
 *************************************************************************/
int leftChild(int index)
{
    // Return the left child index.
    return 2 * index + 1;
}

/*************************************************************************
 * Function: rightChild
 *
 * Description:
 *     Calculates the right child index of a node.
 *************************************************************************/
int rightChild(int index)
{
    // Return the right child index.
    return 2 * index + 2;
}

/*************************************************************************
 * Function: heapifyUp
 *
 * Description:
 *     Moves a newly inserted email upward until the MaxHeap property
 *     is restored.
 *************************************************************************/
void heapifyUp(int index)
{
    // Continue while the current element is not the root.
    while (index > 0)
    {
        // Find the parent of the current element.
        int parentIndex = parent(index);
```

```
        // Stop if the parent already has higher or equal priority.
        if (!heap[index].higherPriorityThan(heap[parentIndex]))
        {
            break;
        }

        // Swap the current email with its parent.
        swap(heap[index], heap[parentIndex]);

        // Continue checking from the parent's old position.
        index = parentIndex;
    }
}

/*****************************************************************************
 * Function: heapifyDown
 *
 * Description:
 *     Moves the root email downward after removing it so that the
 *     MaxHeap property is restored.
 *****************************************************************************/
void heapifyDown(int index)
{
    // Continue checking the current node.
    while (true)
    {
        // Calculate the left child index.
        int left = leftChild(index);

        // Calculate the right child index.
        int right = rightChild(index);

        // Assume the current node has the highest priority.
        int highest = index;

        // Check whether the left child has higher priority.
        if (left < static_cast<int>(heap.size()) &&
            heap[left].higherPriorityThan(heap[highest]))
        {
            // The left child becomes the highest-priority node.
            highest = left;
        }

        // Check whether the right child has higher priority.
```

```
            if (right < static_cast<int>(heap.size()) &&
                heap[right].higherPriorityThan(heap[highest]))
            {
                // The right child becomes the highest-priority node.
                highest = right;
            }

            // If the current node is still highest, the heap is valid.
            if (highest == index)
            {
                break;
            }

            // Swap the current node with the higher-priority child.
            swap(heap[index], heap[highest]);

            // Continue heapifying from the child's old position.
            index = highest;
        }
    }

public:
    /*************************************************************************
     * Function: insert
     *
     * Description:
     *     Adds an email to the MaxHeap.
     ************************************************************************/
    void insert(const Email& email)
    {
        // Add the new email to the end of the array.
        heap.push_back(email);

        // Move the new email upward into its correct position.
        heapifyUp(static_cast<int>(heap.size()) - 1);
    }

    /*************************************************************************
     * Function: peek
     *
     * Description:
     *     Returns the highest-priority email without removing it.
     *
     * Returns:
```

```
 *     A reference to the highest-priority email.
 *****************************************************************************/
const Email& peek() const
{
   // Return the root of the MaxHeap.
   return heap[0];
}

/*****************************************************************************
 * Function: removeMax
 *
 * Description:
 *     Removes the highest-priority email from the MaxHeap.
 *****************************************************************************/
void removeMax()
{
   // Do nothing if the heap is empty.
   if (heap.empty())
   {
      return;
   }

   // If there is only one email, simply remove it.
   if (heap.size() == 1)
   {
      heap.pop_back();
      return;
   }

   // Move the last email to the root position.
   heap[0] = heap.back();

   // Remove the duplicate last element.
   heap.pop_back();

   // Restore the MaxHeap property.
   heapifyDown(0);
}

/*****************************************************************************
 * Function: empty
 *
 * Description:
 *     Determines whether the heap contains any emails.
```

```
     ****************************************************************************/
    bool empty() const
    {
        // Return true when there are no emails.
        return heap.empty();
    }

    /****************************************************************************
     * Function: size
     *
     * Description:
     *     Returns the number of emails in the heap.
     ****************************************************************************/
    int size() const
    {
        // Return the number of emails.
        return static_cast<int>(heap.size());
    }
};

/*******************************************************************************
 * Class: EmailPriorityQueue
 *
 * Description:
 *     Provides the commands needed to manage emails using the MaxHeap.
 *******************************************************************************/
class EmailPriorityQueue
{
private:
    // Stores all unread emails in the custom MaxHeap.
    MaxHeap emailHeap;

    /****************************************************************************
     * Function: processEmail
     *
     * Description:
     *     Parses an EMAIL command and inserts the resulting email.
     ****************************************************************************/
    void processEmail(const string& line)
    {
        // Remove the "EMAIL " portion from the beginning of the command.
        string data = trim(line.substr(6));

        // Find the first comma separating sender and subject.
```

```
        size_t firstComma = data.find(',');

        // Find the second comma separating subject and date.
        size_t secondComma = data.find(',', firstComma + 1);

        // Stop if the expected commas are missing.
        if (firstComma == string::npos ||
            secondComma == string::npos)
        {
            return;
        }

        // Extract and trim the sender category.
        string sender = trim(data.substr(0, firstComma));

        // Extract and trim the subject.
        string subject = trim(
            data.substr(firstComma + 1,
                        secondComma - firstComma - 1)
        );

        // Extract and trim the date.
        string date = trim(data.substr(secondComma + 1));

        // Create an Email object using the parsed fields.
        Email email(sender, subject, date);

        // Add the email to the priority queue.
        emailHeap.insert(email);
    }

public:
    /*************************************************************************
     * Function: processCommand
     *
     * Description:
     *     Determines which command was entered and performs that command.
     *************************************************************************/
    void processCommand(const string& inputLine)
    {
        // Trim whitespace from the input line.
        string line = trim(inputLine);

        // Ignore blank lines.
```

```
    if (line.empty())
    {
        return;
    }

    // Process an EMAIL command.
    if (line.rfind("EMAIL ", 0) == 0)
    {
        processEmail(line);
    }
    // Process a NEXT command.
    else if (line == "NEXT")
    {
        next();
    }
    // Process a READ command.
    else if (line == "READ")
    {
        read();
    }
    // Process a COUNT command.
    else if (line == "COUNT")
    {
        count();
    }
}

/*****************************************************************************
 * Function: next
 *
 * Description:
 *     Displays the highest-priority unread email without removing it.
 *****************************************************************************/
void next()
{
    // Check whether the queue is empty.
    if (emailHeap.empty())
    {
        // Print the required empty-queue message.
        cout << "No emails to read." << endl;

        // Stop because there is no email to display.
        return;
    }
```

```
        // Get the highest-priority email without removing it.
        const Email& email = emailHeap.peek();

        // Print the required NEXT output.
        cout << "Next email:" << endl;

        // Print the sender category.
        cout << "Sender: " << email.getSender() << endl;

        // Print the subject.
        cout << "Subject: " << email.getSubject() << endl;

        // Print the date.
        cout << "Date: " << email.getDate() << endl;
    }

    /*****************************************************************************
     * Function: read
     *
     * Description:
     *     Removes the highest-priority email without displaying it.
     *****************************************************************************/
    void read()
    {
        // Remove the highest-priority email.
        emailHeap.removeMax();
    }

    /*****************************************************************************
     * Function: count
     *
     * Description:
     *     Displays the number of unread emails.
     *****************************************************************************/
    void count()
    {
        // Print the number of unread emails using the required format.
        cout << "There are " << emailHeap.size()
             << " emails to read." << endl;
    }
};

/*****************************************************************************
```

```
 * Class: EmailProgram
 *
 * Description:
 *     Controls the program's file handling and command processing.
 ********************************************************************************/
class EmailProgram
{
private:
   // Stores the name of the input file.
   string filename;

   // Stores the email priority queue.
   EmailPriorityQueue queue;

public:
   /*******************************************************************************
    * Constructor: EmailProgram
    *
    * Description:
    *     Creates the program using the supplied filename.
    *******************************************************************************/
   EmailProgram(const string& inputFilename)
   {
      // Store the input filename.
      filename = inputFilename;
   }

   /*******************************************************************************
    * Function: run
    *
    * Description:
    *     Opens the input file and processes every line.
    *
    * Returns:
    *     0 if the file was successfully processed.
    *     1 if the file could not be opened.
    *******************************************************************************/
   int run()
   {
      // Open the input file.
      ifstream inputFile(filename);

      // Check whether the file opened successfully.
      if (!inputFile.is_open())
```

```
        {
            // Print an error containing the filename.
            cerr << "Error: could not open file " << filename << endl;

            // Return a failure status.
            return 1;
        }

        // Store one line from the input file at a time.
        string line;

        // Read every line, including the final line without a newline.
        while (getline(inputFile, line))
        {
            // Remove a trailing carriage return from Windows CRLF files.
            if (!line.empty() && line.back() == '\r')
            {
                line.pop_back();
            }

            // Process the current command.
            queue.processCommand(line);
        }

        // Close the input file.
        inputFile.close();

        // Return a successful status.
        return 0;
    }
};

/*******************************************************************************
 * Function: main
 *
 * Description:
 *     Creates the program object and starts execution.
 *
 * Parameters:
 *     argc - Number of command-line arguments.
 *     argv - Array containing the command-line arguments.
 *
 * Returns:
 *     Program exit status.
```

```
 ********************************************************************************/
int main(int argc, char* argv[])
{
    // Check that exactly one input filename was supplied.
    if (argc < 2)
    {
        // Print the required usage message.
        cout << "Usage: ./program <input_file>" << endl;

        // Exit because no input file was supplied.
        return 1;
    }

    // Create the main program object using the input filename.
    EmailProgram program(argv[1]);

    // Run the program and return its status.
    return program.run();
}
```

Gemini Output:

```
/*
 * Program Name: EmailPriorityQueue
 * Description: A custom MaxHeap priority queue manager for processing and ordering emails based on sender category and date.
 * Inputs: A text file containing commands (EMAIL, NEXT, READ, COUNT) passed via command-line argument.
 * Outputs: Output to standard console displaying email counts and the next prioritized email details based on the commands.
 * Author: Assistant
 * Creation Date: October 1, 2026
 * Sources: Created with assistance from Google's Gemini AI.
 */

// Include iostream for standard input and output operations
#include <iostream>
// Include string for using the std::string class
#include <string>
// Include vector to act as the dynamic array for our custom heap
#include <vector>
// Include fstream for reading from the input text file
#include <fstream>

// Define the Email class to hold email data and parsing logic
```

```
class Email {
// Public access modifier for the Email fields and methods
public:
    // Store the sender's category as a string
    std::string category;
    // Store the subject line as a string
    std::string subject;
    // Store the date as a string
    std::string dateStr;
    // Store the numeric priority of the category
    int priority;
    // Store the parsed year for sorting
    int year;
    // Store the parsed month for sorting
    int month;
    // Store the parsed day for sorting
    int day;

    // Default constructor for creating empty emails
    Email() {
        // Initialize priority to zero
        priority = 0;
        // Initialize year to zero
        year = 0;
        // Initialize month to zero
        month = 0;
        // Initialize day to zero
        day = 0;
    }

    // Parameterized constructor to initialize an email from parsed strings
    Email(std::string cat, std::string sub, std::string dt) {
        // Assign the category parameter to the class member
        category = cat;
        // Assign the subject parameter to the class member
        subject = sub;
        // Assign the date parameter to the class member
        dateStr = dt;
        // Call the helper method to set the priority integer
        setPriority();
        // Call the helper method to parse the date string into integers
        parseDate();
    }
```

```
    // Method to determine if this email has less priority than another email
    bool isLessThan(const Email& other) const {
        // Check if the categories have different priorities
        if (priority != other.priority) {
            // Return true if this priority is strictly less than the other
            return priority < other.priority;
        }
        // If priorities are equal, check if the years are different
        if (year != other.year) {
            // Return true if this year is strictly less than the other
            return year < other.year;
        }
        // If years are equal, check if the months are different
        if (month != other.month) {
            // Return true if this month is strictly less than the other
            return month < other.month;
        }
        // If months are equal, finally compare the days
        return day < other.day;
    }

// Private access modifier for internal helper methods
private:
    // Method to map the category string to a numeric priority
    void setPriority() {
        // Check if the category is Boss
        if (category == "Boss") {
            // Assign the highest priority of 5
            priority = 5;
        }
        // Check if the category is Subordinate
        else if (category == "Subordinate") {
            // Assign the second highest priority of 4
            priority = 4;
        }
        // Check if the category is Peer
        else if (category == "Peer") {
            // Assign the middle priority of 3
            priority = 3;
        }
        // Check if the category is ImportantPerson
        else if (category == "ImportantPerson") {
            // Assign a lower priority of 2
            priority = 2;
```

```
        }
        // If it's OtherPerson or anything else
        else {
            // Assign the lowest priority of 1
            priority = 1;
        }
    }

    // Method to parse the MM-DD-YYYY string into integer year, month, and day
    void parseDate() {
        // Ensure the string has the correct length before parsing
        if (dateStr.length() >= 10) {
            // Extract the month from index 0, length 2, and convert to integer
            month = std::stoi(dateStr.substr(0, 2));
            // Extract the day from index 3, length 2, and convert to integer
            day = std::stoi(dateStr.substr(3, 2));
            // Extract the year from index 6, length 4, and convert to integer
            year = std::stoi(dateStr.substr(6, 4));
        }
    }
};

// Define the custom MaxHeap class from scratch
class MaxHeap {
// Private access modifier for internal heap data and helper methods
private:
    // The underlying dynamic array holding the heap elements
    std::vector<Email> heap;

    // Helper method to get the parent index of a given index
    int getParent(int i) {
        // Calculate and return the parent index using integer division
        return (i - 1) / 2;
    }

    // Helper method to get the left child index of a given index
    int getLeftChild(int i) {
        // Calculate and return the left child index
        return (2 * i) + 1;
    }

    // Helper method to get the right child index of a given index
    int getRightChild(int i) {
        // Calculate and return the right child index
```

```
        return (2 * i) + 2;
    }

    // Helper method to restore the max-heap property upwards
    void heapifyUp(int index) {
        // Continue looping as long as we aren't at the root and the current node is greater than its
parent
        while (index > 0 && heap[getParent(index)].isLessThan(heap[index])) {
            // Get the parent index
            int parentIndex = getParent(index);
            // Swap the current node with its parent using a temporary variable
            Email temp = heap[index];
            // Move the parent down to the current index
            heap[index] = heap[parentIndex];
            // Move the temporary (original current) to the parent index
            heap[parentIndex] = temp;
            // Update the index to the parent's index to continue checking upwards
            index = parentIndex;
        }
    }

    // Helper method to restore the max-heap property downwards
    void heapifyDown(int index) {
        // Get the current size of the heap
        int size = heap.size();
        // Assume the largest element is the current index
        int largest = index;
        // Get the index of the left child
        int left = getLeftChild(index);
        // Get the index of the right child
        int right = getRightChild(index);

        // Check if the left child exists and is greater than the current largest
        if (left < size && heap[largest].isLessThan(heap[left])) {
            // Update largest to be the left child's index
            largest = left;
        }

        // Check if the right child exists and is greater than the current largest
        if (right < size && heap[largest].isLessThan(heap[right])) {
            // Update largest to be the right child's index
            largest = right;
        }
```

```
        // If the largest is no longer the current index, we need to swap
        if (largest != index) {
            // Swap the current node with the largest child using a temporary variable
            Email temp = heap[index];
            // Move the largest child up to the current index
            heap[index] = heap[largest];
            // Move the temporary (original current) to the largest child index
            heap[largest] = temp;
            // Recursively call heapifyDown on the new location of the node
            heapifyDown(largest);
        }
    }

// Public access modifier for the MaxHeap interface
public:
    // Method to insert a new email into the heap
    void push(const Email& email) {
        // Add the new email to the end of the dynamic array
        heap.push_back(email);
        // Restore the max-heap property starting from the newly added element
        heapifyUp(heap.size() - 1);
    }

    // Method to remove the highest priority email
    void pop() {
        // Check if the heap is empty to avoid popping from an empty vector
        if (heap.empty()) {
            // Return immediately if empty (do nothing)
            return;
        }
        // Overwrite the root (highest priority) with the last element in the array
        heap[0] = heap.back();
        // Remove the last element from the array
        heap.pop_back();
        // If the heap is not empty after popping
        if (!heap.empty()) {
            // Restore the max-heap property starting from the root downwards
            heapifyDown(0);
        }
    }

    // Method to get a pointer to the highest priority email without removing it
    const Email* peek() const {
        // Check if the heap is empty
```

```
        if (heap.empty()) {
            // Return a null pointer if there are no emails
            return nullptr;
        }
        // Return a pointer to the root element
        return &heap[0];
    }

    // Method to get the total number of emails in the heap
    int size() const {
        // Return the size of the underlying vector
        return heap.size();
    }

    // Method to check if the heap is empty
    bool isEmpty() const {
        // Return the boolean result of the vector's empty method
        return heap.empty();
    }
};

// Define a class to handle file reading, parsing, and executing commands
class EmailManager {
// Private access modifier for internal manager state
private:
    // Create an instance of the custom MaxHeap
    MaxHeap priorityQueue;

    // Helper method to remove leading, trailing whitespace and CRLF characters
    std::string trim(const std::string& str) {
        // Find the index of the first non-whitespace character
        size_t start = str.find_first_not_of(" \t\r\n");
        // If the string is entirely whitespace, return an empty string
        if (start == std::string::npos) {
            // Return empty string
            return "";
        }
        // Find the index of the last non-whitespace character
        size_t end = str.find_last_not_of(" \t\r\n");
        // Extract and return the trimmed substring
        return str.substr(start, end - start + 1);
    }

    // Helper method to process an EMAIL addition command
```

```
    void handleEmailCommand(const std::string& data) {
        // Find the position of the first comma separating category and subject
        size_t firstComma = data.find(',');
        // Find the position of the second comma separating subject and date
        size_t secondComma = data.find(',', firstComma + 1);

        // Ensure both commas exist to properly parse the data
        if (firstComma != std::string::npos && secondComma != std::string::npos) {
            // Extract the category string
            std::string category = trim(data.substr(0, firstComma));
            // Extract the subject string
            std::string subject = trim(data.substr(firstComma + 1, secondComma - firstComma - 1));
            // Extract the date string
            std::string date = trim(data.substr(secondComma + 1));

            // Instantiate a new Email object with the parsed fields
            Email newEmail(category, subject, date);
            // Push the new Email object into the custom MaxHeap
            priorityQueue.push(newEmail);
        }
    }

// Public access modifier for the manager interface
public:
    // Main method to run the application logic using command-line arguments
    void run(int argc, char* argv[]) {
        // Check if the filename argument was provided
        if (argc < 2) {
            // Print the usage instructions to standard output
            std::cout << "Usage: ./program <input_file>\n";
            // Exit the function early
            return;
        }

        // Extract the filename from the arguments array
        std::string filename = argv[1];
        // Create an input file stream object
        std::ifstream inputFile(filename);

        // Check if the file failed to open
        if (!inputFile.is_open()) {
            // Print an error message including the problematic filename
            std::cout << "Error: Cannot open file " << filename << "\n";
            // Exit the function early
```

```
    return;
}

// Create a string variable to hold each line read from the file
std::string line;
// Loop continuously, reading line by line until the end of the file is reached
while (std::getline(inputFile, line)) {
    // Trim the whitespace and line endings from the read line
    std::string trimmedLine = trim(line);

    // Skip the iteration if the line was empty or purely whitespace
    if (trimmedLine.empty()) {
        // Continue to the next loop iteration
        continue;
    }

    // Find the first space to isolate the command word
    size_t spacePos = trimmedLine.find(' ');
    // Initialize a string for the command word
    std::string command;
    // Initialize a string for the remainder of the line (data)
    std::string data = "";

    // Check if there is a space in the line
    if (spacePos != std::string::npos) {
        // Extract the command from the beginning to the space
        command = trimmedLine.substr(0, spacePos);
        // Extract the data from the space to the end
        data = trimmedLine.substr(spacePos + 1);
    }
    // If there is no space, the entire line is the command
    else {
        // Assign the full trimmed line to the command
        command = trimmedLine;
    }

    // Check if the command is "EMAIL"
    if (command == "EMAIL") {
        // Pass the data portion to the helper method for processing
        handleEmailCommand(data);
    }
    // Check if the command is "COUNT"
    else if (command == "COUNT") {
        // Print the current size of the priority queue to standard output
```

```
            std::cout << "There are " << priorityQueue.size() << " emails to read.\n";
        }
        // Check if the command is "NEXT"
        else if (command == "NEXT") {
            // Call peek to get the highest priority email
            const Email* nextEmail = priorityQueue.peek();
            // Check if the pointer is null (queue is empty)
            if (nextEmail == nullptr) {
                // Print the empty queue message
                std::cout << "No emails to read.\n";
            }
            // If the pointer is valid
            else {
                // Print the next email header
                std::cout << "Next email:\n";
                // Print the sender category
                std::cout << "Sender: " << nextEmail->category << "\n";
                // Print the email subject
                std::cout << "Subject: " << nextEmail->subject << "\n";
                // Print the email date
                std::cout << "Date: " << nextEmail->dateStr << "\n";
            }
        }
        // Check if the command is "READ"
        else if (command == "READ") {
            // Remove the highest priority email from the heap
            priorityQueue.pop();
        }
    }
    // Close the input file stream to release the resource
    inputFile.close();
  }
};

// Main function entry point for the C++ application
int main(int argc, char* argv[]) {
  // Instantiate the EmailManager object
  EmailManager manager;
  // Call the run method on the manager, passing command line arguments
  manager.run(argc, argv);
  // Return 0 to indicate successful program termination
  return 0;
}
```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

ChatGPT analysis

The ChatGPT code is correct on the sample input and produces the expected output exactly. It takes the filename from argv[1], prints a usage message when the argument is missing, reports an error naming the file if it cannot be opened, and uses getline with \r stripping and trimming, so Windows line endings, a missing trailing newline, and blank lines are all handled. NEXT and READ on an empty queue do not crash. Its weaknesses in correctness are that an unrecognized sender category silently becomes the lowest priority, a malformed date makes stoi throw and crash the program, and emails with the same category and date have no defined order because a heap is not stable. For execution time, insert and removeMax are O(log n) and peek, COUNT, and empty are O(1), so processing a file of n commands is O(n log n). However, dateValue() re-parses the date string with substr and stoi on every comparison, which makes the constant factor larger than necessary. Space complexity is O(n) for the vector of emails, with each Email storing three strings and an int, and removeMax copies a full Email instead of moving it.

For maintainability, the design is strong. Responsibilities are split cleanly across Email, MaxHeap, EmailPriorityQueue, and EmailProgram, fields are private with getters, heapify is iterative, helpers are small, and every line is commented. The main drawbacks are that trim() and main() are free functions, which conflicts with the assignment's requirement to use objects rather than functions, several helper methods are not marked const, unused includes and using namespace std add clutter, unknown commands are ignored without any message, and the prologue lacks collaborators, revision date, and per-block authorship labels.

Gemini analysis

The Gemini code is also correct on the sample input. It reads the filename from argv[1], prints the usage message, reports a file-open error with the filename, and handles CRLF, blank lines, and a missing trailing newline through trim() and getline. Its peek() returns a null pointer on an empty heap, which makes empty NEXT handling safe by design, and READ on an empty queue simply returns. Like ChatGPT, it treats unknown categories as priority 1, can throw from stoi on a bad date, and has no tie-breaker for equal category and date. Its error messages go to cout rather than cerr, and it always exits with status 0. For execution time, push and pop are O(log n) and peek and size are O(1), giving O(n log n) overall. Its advantage is that the date is parsed once in the constructor into integers, so comparisons are cheap. Space is O(n), but the Email stores the date string plus separate year, month, and day ints, and the manual swap through a temporary Email copies three strings each time instead of moving them.

For maintainability, Gemini has no free functions apart from main, which fits the "objects, not functions" requirement better, and main only creates a manager and calls run. However, all Email fields are public, which weakens encapsulation, and run() handles argument checking, file I/O, command parsing, and command execution in one long method, which makes it harder to test or extend. heapifyDown is recursive rather than iterative, helper methods are not const, and the prologue lists "Assistant" as the author and omits collaborators, revision date, and revisions.

f. Select one to use as the basis for this assignment and justify why you picked it.

I selected the ChatGPT code as the base because its structure is more maintainable. Responsibilities are split across four focused classes (Email, MaxHeap, EmailPriorityQueue, EmailProgram), all fields are private with getters, heapifyDown is iterative so it uses O(1) extra space, and swaps use std::swap instead of copying strings through a temporary. It also returns exit codes and sends errors to cerr. Its flaws are small and local (a free trim() function, repeated date parsing), so they can be fixed without restructuring the design. Gemini's strengths, parsing the date once and having no free functions, are both easy to add to the ChatGPT code, while Gemini's weaknesses (public fields, one overloaded run()) would need larger rewrites.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

I made seven improvements to the ChatGPT code. For correctness, I moved trim() from a free function into a StringUtils class as a static method and moved the argument check into EmailProgram, so main() only creates one object and calls run(), which meets the assignment's requirement to use objects rather than functions. I also added an Email::isValidCategory() method so emails with an unknown sender category are skipped with a warning on cerr, instead of being silently treated as the lowest priority, and I made unknown commands print a warning instead of being ignored, which keeps the standard output clean while making bad lines in a test file visible. For execution time and space, insert() now takes the email by value and moves it into the vector, and removeMax() moves the last email into the root with std::move, which avoids copying three strings on every insert and removal. I also removed the unused <algorithm> and <sstream> includes, which reduces clutter and unneeded dependencies. For maintainability, I marked the helper methods parent(), leftChild(), rightChild(), and calculateSenderPriority() as const and replaced using namespace std with std:: qualification, which improves const-correctness and avoids namespace pollution. Finally, I completed the prologue with the author, collaborators, sources, creation and revision dates, and a list of revisions, and I labeled every block of code as written by ChatGPT versus me.